

Parallel and Distributed Systems

Chapter 10: Remote Communication Models

Prof. Dr. Martin Sträßer



Copyright & Acknowledgments

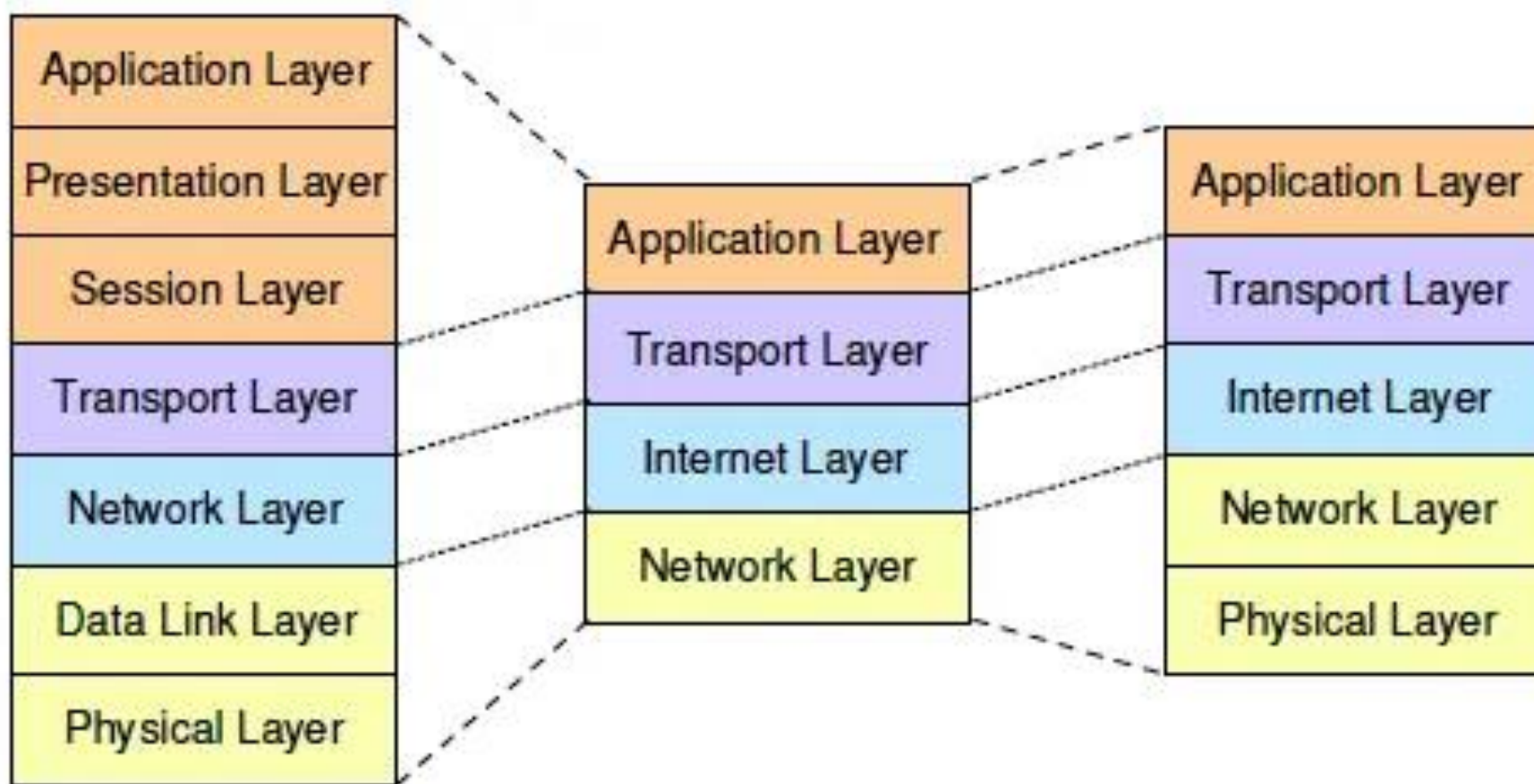
- All materials (incl. lecture and exercise slides, recordings, exercise tasks and sheets, programming tasks) are provided for personal use only. Reproducing, copying, uploading, sharing, or providing materials to third parties is not permitted

Recap: Sockets

- Most widely used and best documented mechanism for message exchange in distributed systems
- Bidirectional communication
- Sender and receiver use sockets
- Steps in working with sockets
 - Creation: Specify properties like communication protocol
 - Binding: Make socket reachable by others
 - Listening: Monitor incoming data at socket
- Sockets are transport layer (ISO/OSI Layer 4) mechanisms



Recap: Layer Models



- Today: Ways to realize communication in distributed systems without being concerned about networking details



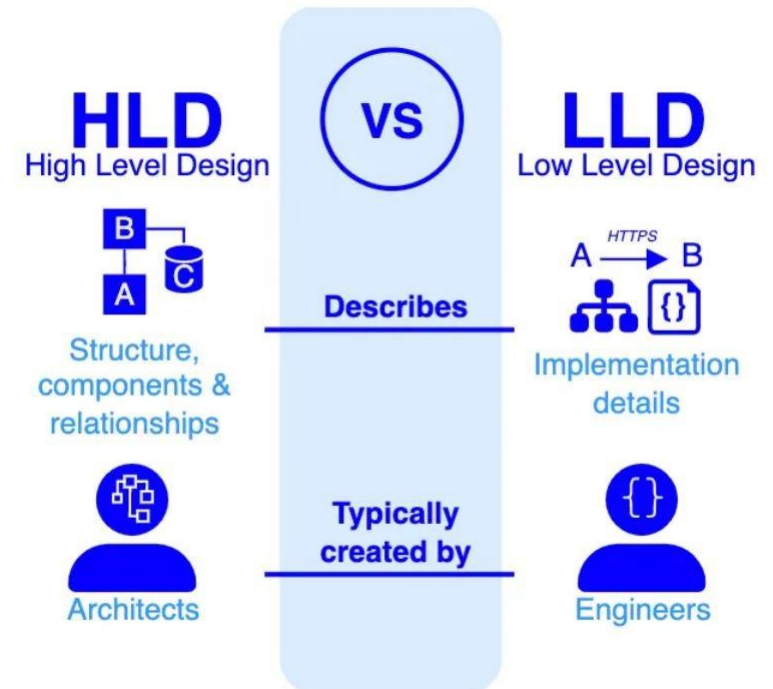
Overview

- Remote Procedure Calls (RPCs)
- The gRPC Protocol
- Representational State Transfer (REST)

Remote Procedure Calls (RPCs)

From Sockets to RPCs

- Shortcomings of Sockets
 - Custom data encoding and decoding needed
 - Potential mismatches between sender and receiver (e.g., Little vs. Big Endian)
 - Programming with sockets is much different than programming local programs
- Solution: Remote Procedure Calls (RPC)
- Idea: Programmers should not be concerned with networking details, communicating with another machine should not be as easy as a method call



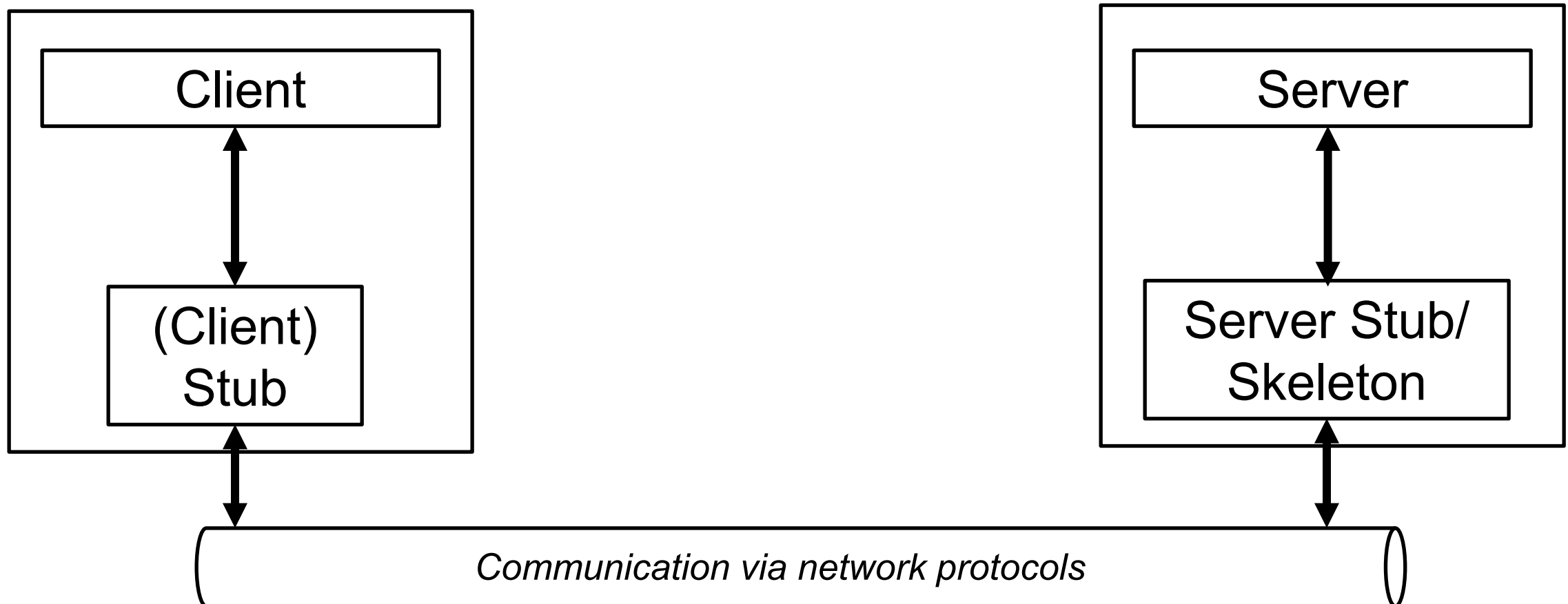


Remote Procedure Calls (RPCs)

- Abstraction from networking
- Much closer to modern OOP-based programming than sockets
- Also well-suited when transforming a local to a distributed application
- Local method call: Object 1 calls a method of Object 2, both are in the same process/address space
- Remote method call: Object 1 calls a method of Object 2, they are in different processes/address spaces, potentially on different machines
- Often synchronous communication (caller waits for response)

Stubs and Skeletons

- Concept of stubs and skeletons (also: client and server stubs) hides distributed nature from programmer





Stubs and Skeletons

- Server holds “real” object of interest
- Client holds a stub that offers the exact same methods as the server object
- Client interacts with the stub as if it was the “real” object
- Every method call on the stub creates network requests to the server
- The server stub/skeleton is the receiving module at the server
- It receives the network requests and executes the method call on the “real” object
- Server stub forwards return values or exceptions to client



Stubs and Skeletons

- Programmers do not directly develop stubs and skeletons
- Skeletons and stubs are generated automatically
- Steps when developing a RPC application
 - Defining the interface: List methods with their parameter and return types using an **interface definition language (IDL)**
 - Implement the interface: Implement a class that has all methods from the interface
 - Object creation: Obviously, the server should create one or multiple objects of the implemented class. It may register the objects at a registry.
 - Connecting the client: The client needs to know the address of the server and relevant object names. The client may query a registry for the desired object and creates a stub if it is there.

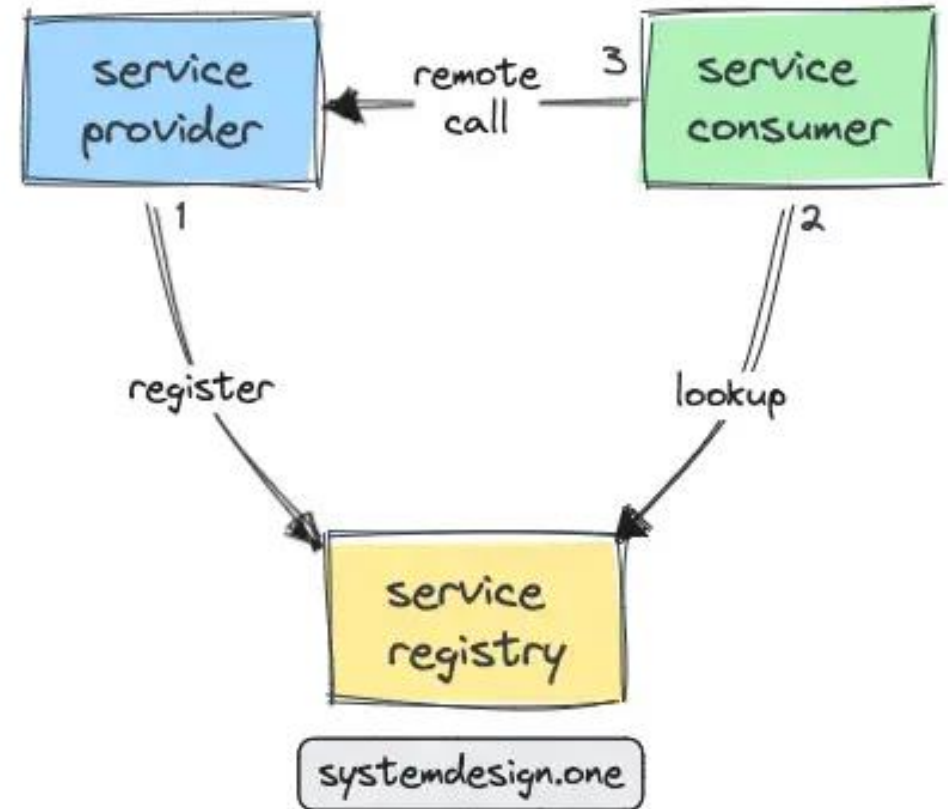


Registry

- Under the hood, RPC also use network messaging
- Clients need to know how to reach servers (IP address, ports, etc.)
- Option 1: Broadcast messages
 - Inside a local network, a client may send a broadcast message to find the server
- Option 2: Registry
 - Servers and clients know the fixed address of a registry
 - Servers register objects at the registry
 - Client queries connection information from registry

Registry

- Important operations
 - Bind: Register new object with a new name
 - Rebind: Make a known name target a new object
 - Unbind: Delete an object
 - Lookup: Get object linked with provided name
 - List: List all known names
- Can be standalone software or integrated into the application code



Parallel RPC

- What if two or more clients access a remote object at the same time?
- Typically, both stubs and skeletons support concurrency
- Often, all calls are handled in separate threads
- Synchronization not by default, has to be implemented by the programmer





RPC Parameters and Return Values

- Methods usually have parameters and/or return values
- Also supported in RPC
- Transportation of parameters and return values via network requires serialization and deserialization
- Every complex data structure is recursively broken and serialized into primitives (numbers and strings)
- Most formats represent key-value mappings
- Example formats: JSON and YAML



RPC Parameters and Return Values

Python class and object

```
class MovieCollection: 1 usage
    def __init__(self, owner: str, year: int, movies: list[str]):
        self.owner = owner
        self.year = year
        self.movies = movies

collection = MovieCollection(
    owner="Alice",
    year=2024,
    movies=["Dune", "Neuromancer", "The Left Hand of Darkness"]
)
```

See movie example

RPC Parameters and Return Values

JSON

```
{  
  "owner": "Alice",  
  "year": 2024,  
  "movies": [  
    "Dune",  
    "Neuromancer",  
    "The Left Hand of Darkness"  
  ]  
}
```

YAML

```
owner: Alice  
year: 2024  
movies:  
  - Dune  
  - Neuromancer  
  - The Left Hand of Darkness
```

See movie example